

Министерство образования и науки Российской Федерации
федеральное государственное автономное образовательное учреждение высшего образования

**“ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО”**

Факультет прикладной информатики (фПИИ)

Направление подготовки (специальность) – 45.03.04 (Языковые модели и
Искусственный Интеллект)

Алгоритмы и структуры данных

Лабораторная работа № 3

Вариант 21

Выполнил студент: Попов Глеб Ильич Группа № К3260

Преподаватель: Харлунин Александр Александрович

г. Санкт-Петербург 2026 г

Задача №1. Лабиринт	3
Задача №2. Компоненты	7
Задача №3. Циклы	11
Задача №4. Порядок курсов	15
Задача №5. Город с односторонним движением	19
Задача №7. Двудольный граф	24
Задача №8. Стоимость полета	29
Задача №9. Аномалии курсов валют	33
Задача №10. Оптимальный обмен валюты	37
Задача №11. Алхимия	43
Общий вывод по работе	48

Задача №1. Лабиринт

Текст задачи:

Лабиринт можно представить в виде неориентированного графа. Вершины графа соответствуют клеткам лабиринта, а ребра показывают, что между двумя соседними клетками можно пройти.

Дан неориентированный граф с n вершинами и m ребрами, а также две различные вершины u и v . Необходимо проверить, существует ли путь между вершинами u и v .

Если путь существует, нужно вывести 1, иначе вывести 0.

Код:

```
import time

import tracemalloc

from collections import deque

def solve():

    with open("input.txt", "r", encoding="utf-8") as f:

        data = list(map(int, f.read().split()))

    n = data[0]

    m = data[1]

    graph = [[] for _ in range(n + 1)]

    pos = 2
```

```
for _ in range(m):

    a = data[pos]

    b = data[pos + 1]

    pos += 2

    graph[a].append(b)

    graph[b].append(a)

start = data[pos]

finish = data[pos + 1]

used = [False] * (n + 1)

q = deque()

q.append(start)

used[start] = True

while q:

    v = q.popleft()

    if v == finish:

        with open("output.txt", "w", encoding="utf-8") as f:

            f.write("1\n")

        return

    for to in graph[v]:

        if not used[to]:
```

```

        used[to] = True

        q.append(to)

    with open("output.txt", "w", encoding="utf-8") as f:

        f.write("0\n")

    tracemalloc.start()

    start_time = time.perf_counter()

    solve()

    finish_time = time.perf_counter()

    cur, peak = tracemalloc.get_traced_memory()

    tracemalloc.stop()

    with open("stats.txt", "w", encoding="utf-8") as f:

        f.write(f"time = {finish_time - start_time:.6f} sec\n")

        f.write(f"memory = {peak / 1024:.2f} KB\n")

```

Текстовое объяснение решения: В этой задаче необходимо определить, можно ли добраться из одной вершины графа в другую.

Для этого используется поиск в ширину - BFS. Сначала граф считывается в виде списка смежности. Так как граф неориентированный, каждое ребро добавляется в обе стороны: из a в b и из b в a.

После этого создается массив `used`, в котором отмечаются уже посещенные вершины. В очередь добавляется стартовая вершина. Далее, пока очередь не пуста, из нее достается очередная вершина. Если эта вершина совпадает с конечной, значит путь найден, и программа выводит 1.

Если вершина не является конечной, в очередь добавляются все ее непосещенные соседи. Если обход закончился, но конечная вершина так и не была найдена, значит пути между вершинами нет, и программа выводит 0.

Так как каждая вершина и каждое ребро обрабатываются не более одного раза, алгоритм работает за линейное время относительно размера графа.

Таблица тестирования:

время	память	input	output
0.001492 sec	11.34 KB	4 4 1 2 3 2 4 3 1 4 1 4	1
0.001466 sec	11.25 KB	4 2 1 2 3 2 1 4	0
0.001822 sec	11.38 KB	5 3 1 2 2 3 4 5 1 3	1
0.001179 sec	11.39 KB	6 3 1 2 2 3 5 6 1 6	0

Вывод по задаче

В ходе решения задачи был реализован поиск пути между двумя вершинами неориентированного графа. Для обхода графа использовался алгоритм BFS, который позволяет последовательно посещать все вершины, достижимые из стартовой. Программа корректно определяет наличие и отсутствие пути между заданными вершинами и укладывается в ограничения по времени и памяти.

Задача №2. Компоненты

Текст задачи:

Дан неориентированный граф с n вершинами и m ребрами. Нужно посчитать количество компонент связности в этом графе.

Компонента связности - это такая часть графа, внутри которой из любой вершины можно добраться до любой другой вершины по ребрам графа. Если вершина не соединена ни с одной другой вершиной, она сама образует отдельную компоненту связности.

Код:

```
import time

import tracemalloc

from collections import deque

def solve():

    with open("input.txt", "r", encoding="utf-8") as f:

        data = list(map(int, f.read().split()))
```

```
n = data[0]

m = data[1]

graph = [[] for _ in range(n + 1)]

pos = 2

for _ in range(m):
    a = data[pos]
    b = data[pos + 1]
    pos += 2

    graph[a].append(b)
    graph[b].append(a)

used = [False] * (n + 1)
components = 0

for i in range(1, n + 1):
    if not used[i]:
        components += 1

        q = deque()
        q.append(i)
        used[i] = True

        while q:
```



```
        v = q.popleft()

        for to in graph[v]:

            if not used[to]:

                used[to] = True

                q.append(to)

    with open("output.txt", "w", encoding="utf-8") as f:

        f.write(str(components) + "\n")

tracemalloc.start()

start_time = time.perf_counter()

solve()

finish_time = time.perf_counter()

cur, peak = tracemalloc.get_traced_memory()

tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:

    f.write(f"time = {finish_time - start_time:.6f} sec\n")

    f.write(f"memory = {peak / 1024:.2f} KB\n")
```

Текстовое объяснение решения: В этой задаче необходимо определить, на сколько отдельных связных частей разбивается неориентированный граф.

Для решения используется поиск в ширину. Сначала граф считывается в виде списка смежности. Поскольку граф неориентированный, каждое ребро добавляется в обе стороны.

Затем создается массив `used`, который показывает, была ли вершина уже посещена. После этого программа перебирает все вершины от 1 до `n`. Если очередная вершина еще не была посещена, значит найдена новая компонента связности. Счетчик компонент увеличивается на единицу, а из этой вершины запускается BFS.

Во время BFS посещаются все вершины, которые достижимы из текущей стартовой вершины. Все они относятся к одной компоненте связности. После завершения обхода программа продолжает перебирать вершины. Если встречается новая не посещенная вершина, начинается обход следующей компоненты.

В конце программа выводит количество найденных компонент связности.

Таблица тестирования:

время	память	input	output
0.001419 sec	11.25 KB	4 2 1 2 3 2	2
0.002524 sec	11.16 KB	5 0	5
0.001472 sec	11.42 KB	6 4 1 2 2 3 4 5 5 6	2
0.001272 sec	11.34 KB	7 3	4

		1 2 2 3 6 7	
--	--	-------------------	--

Вывод по задаче

В ходе решения задачи был реализован подсчет компонент связности в неориентированном графе. Для каждой еще не посещенной вершины запускается поиск в ширину, который помечает всю соответствующую компоненту. Такой подход позволяет корректно обработать как связные графы, так и графы с изолированными вершинами. Алгоритм работает эффективно, так как каждая вершина и каждое ребро рассматриваются не более одного раза.

Задача №3. Циклы

Текст задачи:

Дан ориентированный граф с n вершинами и m ребрами. Необходимо определить, содержит ли этот граф цикл.

Граф можно представить как набор зависимостей между вершинами. Если в графе существует путь, по которому можно вернуться в уже обрабатываемую вершину, значит в графе есть цикл.

Если цикл есть, нужно вывести 1, если цикла нет - вывести 0

Код:

```
import time
import tracemalloc

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = list(map(int, f.read().split()))

    n = data[0]
    m = data[1]
```

```

graph = [[] for _ in range(n + 1)]

pos = 2

for _ in range(m):
    a = data[pos]
    b = data[pos + 1]
    pos += 2

    graph[a].append(b)

color = [0] * (n + 1)

for start in range(1, n + 1):
    if color[start] != 0:
        continue

    stack = [(start, 0)]

    while stack:
        v, index = stack[-1]

        if color[v] == 0:
            color[v] = 1

        if index == len(graph[v]):
            color[v] = 2
            stack.pop()
            continue

        to = graph[v][index]
        stack[-1] = (v, index + 1)

        if color[to] == 0:
            stack.append((to, 0))
        elif color[to] == 1:
            with open("output.txt", "w", encoding="utf-8") as
f:
                f.write("1\n")
            return

    with open("output.txt", "w", encoding="utf-8") as f:
        f.write("0\n")

tracemalloc.start()
start_time = time.perf_counter()

```

```

solve()

finish_time = time.perf_counter()
cur, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:
    f.write(f"time = {finish_time - start_time:.6f} sec\n")
    f.write(f"memory = {peak / 1024:.2f} KB\n")

```

Текстовое объяснение решения: В этой задаче нужно проверить наличие цикла в ориентированном графе.

Для решения используется обход в глубину. Чтобы избежать проблем с глубиной рекурсии, обход реализован итеративно через стек. Для каждой вершины хранится цвет:

- 0 - вершина еще не посещалась;
- 1 - вершина сейчас находится в обработке;
- 2 - вершина полностью обработана.

Если во время обхода из текущей вершины находится ребро в вершину цвета 1, значит мы пришли в вершину, которая уже находится в текущем пути обхода. Это означает, что в графе есть цикл, поэтому программа сразу выводит 1.

Если все вершины были обработаны и такого ребра не встретилось, то циклов в графе нет. В этом случае программа выводит 0.

Такой способ корректно работает и для несвязных ориентированных графов, потому что запуск обхода производится из каждой еще не посещенной вершины.

Таблица тестирования:

время	память	input	output
-------	--------	-------	--------

0.002247 sec	10.70 KB	4 4 1 2 4 1 2 3 3 1	1
0.001900 sec	10.58 KB	5 7 1 2 2 3 1 3 3 4 1 4 2 5 3 5	0
0.001069 sec	10.56 KB	3 3 1 2 2 3 3 1	1
0.001162 sec	10.48 KB	4 2 1 2 3 4	0

Вывод по задаче

В ходе решения задачи был реализован алгоритм поиска цикла в ориентированном графе. Для этого использовался обход в глубину с раскраской вершин в три состояния. Программа корректно определяет наличие обратного ребра в текущем пути обхода, что является признаком цикла. Решение работает без рекурсии, поэтому оно устойчиво даже при большой глубине графа.

Задача №4. Порядок курсов

Текст задачи:

Дан ориентированный ациклический граф с n вершинами и m ребрами. Нужно выполнить топологическую сортировку графа.

Топологическая сортировка - это такой порядок вершин, при котором для каждого ребра $a \rightarrow b$ вершина a находится раньше вершины b .

Если существует несколько правильных порядков, можно вывести любой из них.

Код:

```
import time

import tracemalloc

from collections import deque


def solve():

    with open("input.txt", "r", encoding="utf-8") as f:

        data = list(map(int, f.read().split()))

    n = data[0]

    m = data[1]

    graph = [[] for _ in range(n + 1)]

    indeg = [0] * (n + 1)

    pos = 2

    for _ in range(m):

        a = data[pos]

        b = data[pos + 1]

        pos += 2
```

```

graph[a].append(b)

indeg[b] += 1

q = deque()

for i in range(1, n + 1):
    if indeg[i] == 0:
        q.append(i)

answer = []

while q:
    v = q.popleft()
    answer.append(v)

    for to in graph[v]:
        indeg[to] -= 1

        if indeg[to] == 0:
            q.append(to)

with open("output.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(map(str, answer)) + "\n")

tracemalloc.start()

start_time = time.perf_counter()

```



```
solve()

finish_time = time.perf_counter()

cur, peak = tracemalloc.get_traced_memory()

tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:

    f.write(f"time = {finish_time - start_time:.6f} sec\n")

    f.write(f"memory = {peak / 1024:.2f} KB\n")
```

Текстовое объяснение решения: В этой задаче необходимо построить порядок прохождения курсов так, чтобы все зависимости были выполнены. Если есть ребро $a \rightarrow b$, это означает, что вершина a должна идти раньше вершины b .

Для решения используется алгоритм Кана. Сначала для каждой вершины считается входящая степень - количество ребер, которые входят в эту вершину. Все вершины с входящей степенью 0 можно поставить в начало порядка, так как перед ними нет обязательных зависимостей.

Такие вершины помещаются в очередь. Далее из очереди достается вершина, добавляется в ответ, после чего для всех ее соседей уменьшается входящая степень. Если у какой-либо вершины входящая степень стала равна 0, значит все ее зависимости уже обработаны, и ее можно добавить в очередь.

Так как по условию граф является ациклическим, в результате все вершины будут добавлены в ответ. Полученный порядок и является топологической сортировкой.

Таблица тестирования:

время	память	input	output
0.001738 sec	11.28 KB	4 3 1 2 4 1 3 1	3 4 1 2
0.001146 sec	11.22 KB	4 1 3 1	2 3 4 1
0.002022 sec	11.41 KB	5 7 2 1 3 2 3 1 4 3 4 1 5 2 5 3	4 5 3 2 1
0.001600 sec	11.42 KB	6 5 1 3 2 3 3 4 3 5 5 6	1 2 3 4 5 6

Вывод по задаче

В ходе решения задачи был реализован алгоритм топологической сортировки ориентированного ациклического графа. Для каждой вершины была посчитана входящая степень, после чего вершины без зависимостей последовательно добавлялись в ответ. Решение корректно строит порядок вершин, при котором каждое ребро направлено от более ранней вершины к более поздней. Алгоритм работает эффективно, так как каждая вершина и каждое ребро обрабатываются один раз.

Задача №5. Город с односторонним движением

Текст задачи:

Дан ориентированный граф с n вершинами и m ребрами. Необходимо вычислить количество компонент сильной связности.

Компонента сильной связности - это такая группа вершин ориентированного графа, в которой из каждой вершины можно добраться до любой другой вершины этой же группы.

Нужно вывести одно число - количество компонент сильной связности в графе.

Код:

```
import time

import tracemalloc

def solve():

    with open("input.txt", "r", encoding="utf-8") as f:

        data = list(map(int, f.read().split()))

    n = data[0]

    m = data[1]

    graph = [[] for _ in range(n + 1)]

    reverse_graph = [[] for _ in range(n + 1)]

    pos = 2

    for _ in range(m):
```

```
a = data[pos]

b = data[pos + 1]

pos += 2


graph[a].append(b)

reverse_graph[b].append(a)


used = [False] * (n + 1)

order = []


for start in range(1, n + 1):

    if used[start]:

        continue


    stack = [(start, 0)]

    used[start] = True


    while stack:

        v, index = stack[-1]


        if index == len(graph[v]):

            order.append(v)

            stack.pop()

            continue


        to = graph[v][index]

        stack[-1] = (v, index + 1)
```

```
        if not used[to]:
            used[to] = True
            stack.append((to, 0))

used = [False] * (n + 1)
components = 0

for start in reversed(order):
    if used[start]:
        continue

    components += 1
    stack = [start]
    used[start] = True

    while stack:
        v = stack.pop()

        for to in reverse_graph[v]:
            if not used[to]:
                used[to] = True
                stack.append(to)

with open("output.txt", "w", encoding="utf-8") as f:
    f.write(str(components) + "\n")
```

```
tracemalloc.start()

start_time = time.perf_counter()

solve()

finish_time = time.perf_counter()

cur, peak = tracemalloc.get_traced_memory()

tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:

    f.write(f"time = {finish_time - start_time:.6f} sec\n")

    f.write(f"memory = {peak / 1024:.2f} KB\n")
```

Текстовое объяснение решения: В этой задаче необходимо найти количество компонент сильной связности в ориентированном графе.

Для решения используется алгоритм Косарайю. Сначала строятся два графа: исходный граф и обратный граф, в котором все ребра направлены в противоположную сторону.

На первом этапе выполняется обход исходного графа в глубину. Вершины добавляются в массив `order` после полной обработки всех исходящих из них ребер. Таким образом получается порядок выхода из вершин.

На втором этапе вершины рассматриваются в обратном порядке массива `order`. Для каждой еще не посещенной вершины запускается обход по обратному графу. Все вершины, которые будут достигнуты во время этого обхода, образуют одну компоненту сильной связности. После завершения такого обхода счетчик компонент увеличивается.

Рекурсия в решении не используется. Оба обхода реализованы через стек, поэтому программа не зависит от ограничения глубины рекурсии.

Таблица тестирования:

время	память	input	output
0.001814 sec	10.90 KB	4 4 1 2 4 1 2 3 3 1	2
0.001095 sec	11.05 KB	5 7 2 1 3 2 3 1 4 3 4 1 5 2 5 3	5
0.001235 sec	10.62 KB	3 3 1 2 2 3 3 1	1
0.001421 sec	11.26 KB	6 5 1 2 2 1 3 4 4 3 6 5	4

Вывод по задаче

В ходе решения задачи был реализован алгоритм поиска компонент сильной связности в ориентированном графе. Сначала был выполнен обход исходного графа для получения порядка обработки вершин, затем обход обратного графа для выделения самих компонент. Решение корректно работает как для полностью сильносвязного графа, так и для

графа, состоящего из нескольких отдельных компонент. Алгоритм обрабатывает каждую вершину и каждое ребро линейное число раз.

Задача №7. Двудольный граф

Текст задачи:

Неориентированный граф называется двудольным, если его вершины можно разбить на две части так, что каждое ребро соединяет вершины из разных частей.

Другими словами, граф является двудольным, если его вершины можно раскрасить в два цвета так, чтобы концы каждого ребра имели разные цвета.

Дан неориентированный граф с n вершинами и m ребрами. Нужно проверить, является ли он двудольным. Если граф двудольный, нужно вывести 1, иначе вывести 0

Код:

```
import time

import tracemalloc

from collections import deque

def solve():

    with open("input.txt", "r", encoding="utf-8") as f:

        data = list(map(int, f.read().split()))

    n = data[0]

    m = data[1]
```



```
graph = [[] for _ in range(n + 1)]
```

```
pos = 2
```

```
for _ in range(m):
```

```
    a = data[pos]
```

```
    b = data[pos + 1]
```

```
    pos += 2
```

```
    graph[a].append(b)
```

```
    graph[b].append(a)
```

```
color = [-1] * (n + 1)
```

```
for start in range(1, n + 1):
```

```
    if color[start] != -1:
```

```
        continue
```

```
    color[start] = 0
```

```
    q = deque()
```

```
    q.append(start)
```

```
    while q:
```

```
        v = q.popleft()
```

```
        for to in graph[v]:
```

```
            if color[to] == -1:
```

```

        color[to] = 1 - color[v]

        q.append(to)

    elif color[to] == color[v]:

        with open("output.txt", "w", encoding="utf-8") as
f:

            f.write("0\n")

        return

    with open("output.txt", "w", encoding="utf-8") as f:

        f.write("1\n")

tracemalloc.start()

start_time = time.perf_counter()

solve()

finish_time = time.perf_counter()

cur, peak = tracemalloc.get_traced_memory()

tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:

    f.write(f"time = {finish_time - start_time:.6f} sec\n")

    f.write(f"memory = {peak / 1024:.2f} KB\n")

```

Текстовое объяснение решения: В этой задаче нужно проверить, можно ли раскрасить вершины неориентированного графа в два цвета так, чтобы каждое ребро соединяло вершины разных цветов.

Для решения используется поиск в ширину. Сначала граф считывается в виде списка смежности. Так как граф неориентированный, каждое ребро добавляется в обе стороны.

Для каждой вершины хранится цвет в массиве color. Значение -1 означает, что вершина еще не была раскрашена. Если очередная вершина не раскрашена, ей присваивается цвет 0, после чего из нее запускается BFS.

Во время обхода для каждого соседа проверяется его цвет. Если сосед еще не раскрашен, ему присваивается противоположный цвет. Если сосед уже раскрашен тем же цветом, что и текущая вершина, значит граф не является двудольным, и программа выводит 0.

Если весь граф удалось раскрасить без конфликтов, программа выводит 1.

Так как граф может состоять из нескольких компонент связности, обход запускается из каждой еще не раскрашенной вершины.

Таблица тестирования:

время	память	input	output
0.002018 sec	11.42 KB	4 4 1 2 4 1 2 3 3 1	0
0.002212 sec	11.38 KB	5 4 5 2 4 2 3 4 1 4	1
0.001571 sec	11.12 KB	3 0	1

0.001320 sec	11.42 KB	4 5 1 2 2 3 3 4 4 1 1 3	0
--------------	----------	--	---

Вывод по задаче

В ходе решения задачи была реализована проверка неориентированного графа на двудольность. Для этого использовался поиск в ширину и раскраска вершин в два цвета. Если при обходе обнаруживается ребро между вершинами одного цвета, граф не является двудольным. Программа корректно работает как для связных, так и для несвязных графов и укладывается в ограничения по времени и памяти.

Задача №8. Стоимость полета

Текст задачи:

Дан ориентированный взвешенный граф с n вершинами и m ребрами. Вес ребра означает стоимость перелета из одного города в другой.

Также даны две вершины u и v . Нужно найти минимальную общую стоимость пути из вершины u в вершину v .

Если добраться из u в v невозможно, нужно вывести -1.

По условию все веса ребер неотрицательные, поэтому для решения удобно использовать алгоритм Дейкстры.

Код:

```
import time
import tracemalloc
import heapq
```

```
def solve():  
    with open("input.txt", "r", encoding="utf-8") as f:  
        data = list(map(int, f.read().split()))  
  
    n = data[0]  
    m = data[1]  
  
    graph = [[] for _ in range(n + 1)]  
  
    pos = 2  
  
    for _ in range(m):  
        a = data[pos]  
        b = data[pos + 1]  
        w = data[pos + 2]  
        pos += 3  
  
        graph[a].append((b, w))  
  
    start = data[pos]  
    finish = data[pos + 1]  
  
    inf = 10 ** 30  
    dist = [inf] * (n + 1)  
    dist[start] = 0
```

```

heap = []

heapq.heappush(heap, (0, start))

while heap:

    cur_dist, v = heapq.heappop(heap)

    if cur_dist != dist[v]:

        continue

    for to, weight in graph[v]:

        new_dist = cur_dist + weight

        if new_dist < dist[to]:

            dist[to] = new_dist

            heapq.heappush(heap, (new_dist, to))

with open("output.txt", "w", encoding="utf-8") as f:

    if dist[finish] == inf:

        f.write("-1\n")

    else:

        f.write(str(dist[finish]) + "\n")

tracemalloc.start()

start_time = time.perf_counter()

solve()

```

```
finish_time = time.perf_counter()

cur, peak = tracemalloc.get_traced_memory()

tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:

    f.write(f"time = {finish_time - start_time:.6f} sec\n")

    f.write(f"memory = {peak / 1024:.2f} KB\n")
```

Текстовое объяснение решения: В этой задаче нужно найти кратчайший путь во взвешенном ориентированном графе. Так как веса ребер неотрицательные, используется алгоритм Дейкстры.

Сначала граф считывается в виде списка смежности. Для каждой вершины хранится список пар: соседняя вершина и стоимость перелета до нее.

Затем создается массив `dist`, где `dist[i]` - минимальная найденная стоимость пути из стартовой вершины в вершину `i`. Изначально все расстояния равны бесконечности, а расстояние до стартовой вершины равно 0.

Для быстрого выбора вершины с минимальным текущим расстоянием используется приоритетная очередь `heap`. На каждом шаге из очереди достается вершина с наименьшей стоимостью пути. После этого просматриваются все исходящие из нее ребра. Если через текущую вершину можно улучшить расстояние до соседней вершины, значение обновляется и новая пара добавляется в очередь.

После завершения алгоритма проверяется расстояние до конечной вершины. Если оно осталось равным бесконечности, значит пути нет, и выводится -1. Иначе выводится найденная минимальная стоимость.

Таблица тестирования:

время	память	input	output
0.002358 sec	10.57 KB	4 4 1 2 1 4 1 2 2 3 2 1 3 5 1 3	3
0.001703 sec	10.84 KB	5 9 1 2 4 1 3 2 2 3 2 3 2 1 2 4 2 3 5 4 5 4 1 2 5 3 3 4 4 1 5	6
0.001260 sec	10.50 KB	3 3 1 2 7 1 3 5 2 3 2 3 2	-1
0.001440 sec	10.63 KB	4 5 1 2 10 1 3 1 3 2 2 2 4 3 3 4 10 1 4	6

Вывод по задаче

В ходе решения задачи был реализован поиск кратчайшего пути во взвешенном ориентированном графе. Для этого использовался алгоритм Дейкстры с приоритетной очередью. Такой подход позволяет эффективно находить минимальную стоимость перелета при неотрицательных весах ребер. Программа также корректно обрабатывает случай, когда конечная вершина недостижима из начальной.

Задача №9. Аномалии курсов валют

Текст задачи:

Дан ориентированный взвешенный граф с n вершинами и m ребрами. Вес ребра может быть отрицательным.

Необходимо проверить, содержит ли граф цикл с отрицательным суммарным весом. Если такой цикл существует, нужно вывести 1, иначе вывести 0.

В задаче это связано с обменом валют: если произведение курсов обмена по циклу больше единицы, то после последовательности обменов можно получить больше исходной валюты, чем было изначально. После преобразования весов через $-\log$ такая ситуация сводится к поиску отрицательного цикла в графе.

Код:

```
import time

import tracemalloc

def solve():

    with open("input.txt", "r", encoding="utf-8") as f:

        data = list(map(int, f.read().split()))

    n = data[0]

    m = data[1]

    edges = []

    pos = 2
```

```
for _ in range(m):

    a = data[pos]

    b = data[pos + 1]

    w = data[pos + 2]

    pos += 3

    edges.append((a, b, w))

dist = [0] * (n + 1)

for i in range(n):

    changed = False

    for a, b, w in edges:

        if dist[b] > dist[a] + w:

            dist[b] = dist[a] + w

            changed = True

    if not changed:

        with open("output.txt", "w", encoding="utf-8") as f:

            f.write("0\n")

        return

with open("output.txt", "w", encoding="utf-8") as f:

    f.write("1\n")
```

```
tracemalloc.start()

start_time = time.perf_counter()

solve()

finish_time = time.perf_counter()

cur, peak = tracemalloc.get_traced_memory()

tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:

    f.write(f"time = {finish_time - start_time:.6f} sec\n")

    f.write(f"memory = {peak / 1024:.2f} KB\n")
```

Текстовое объяснение решения: В этой задаче нужно определить, есть ли в ориентированном взвешенном графе отрицательный цикл.

Для решения используется алгоритм Беллмана-Форда. Обычно этот алгоритм ищет кратчайшие расстояния от одной стартовой вершины, но здесь нужно проверить отрицательный цикл в любой части графа. Поэтому расстояния до всех вершин изначально задаются равными 0. Это позволяет как будто начать обход сразу из всех вершин.

Далее выполняется n итераций релаксации всех ребер. Если на очередной итерации для ребра $a \rightarrow b$ можно улучшить расстояние до вершины b , то значение $dist[b]$ обновляется.

Если на какой-то итерации изменений не произошло, значит дальнейших улучшений нет и отрицательного цикла в графе не существует. В этом случае программа выводит 0.

Если же улучшение произошло на n -й итерации, это означает, что в графе есть цикл с отрицательной суммой весов. Тогда программа выводит 1.

Таблица тестирования:

время	память	input	output
0.001770 sec	10.54 KB	4 4 1 2 -5 4 1 2 2 3 2 3 1 1	1
0.001622 sec	10.47 KB	3 3 1 2 1 2 3 1 1 3 5	0
0.001412 sec	10.47 KB	3 3 1 2 2 2 3 -4 3 1 1	1
0.001506 sec	10.41 KB	4 2 1 2 -1 3 4 -1	0

Вывод по задаче

В ходе решения задачи был реализован поиск отрицательного цикла в ориентированном взвешенном графе. Для этого использовался алгоритм Беллмана-Форда с начальными расстояниями, равными нулю для всех вершин. Такой подход позволяет проверить наличие отрицательного цикла не только из одной стартовой вершины, а во всем графе. Программа корректно определяет как наличие, так и отсутствие отрицательного цикла.

Задача №10. Оптимальный обмен валюты

Текст задачи:

Дан ориентированный взвешенный граф с n вершинами и m ребрами. Веса ребер могут быть отрицательными. Также задана стартовая вершина s .

Необходимо вычислить длины кратчайших путей из вершины s во все остальные вершины графа.

Для каждой вершины нужно вывести:

*, если пути из s в эту вершину нет;

-, если путь из s в вершину существует, но кратчайшего пути нет, то есть расстояние можно сделать бесконечно маленьким из-за достижимого отрицательного цикла;

длину кратчайшего пути во всех остальных случаях..

Код:

```
import time

import tracemalloc

from collections import deque

def solve():

    with open("input.txt", "r", encoding="utf-8") as f:

        data = list(map(int, f.read().split()))

    n = data[0]

    m = data[1]

    edges = []
```

```
graph = [[] for _ in range(n + 1)]

pos = 2

for _ in range(m):
    a = data[pos]
    b = data[pos + 1]
    w = data[pos + 2]
    pos += 3

    edges.append((a, b, w))
    graph[a].append(b)

start = data[pos]

inf = 10 ** 30
dist = [inf] * (n + 1)
dist[start] = 0

for _ in range(n - 1):
    changed = False

    for a, b, w in edges:
        if dist[a] != inf and dist[b] > dist[a] + w:
            dist[b] = max(-inf, dist[a] + w)
            changed = True
```

```

        if not changed:

            break

bad = [False] * (n + 1)

q = deque()

for a, b, w in edges:

    if dist[a] != inf and dist[b] > dist[a] + w:

        if not bad[b]:

            bad[b] = True

            q.append(b)

while q:

    v = q.popleft()

    for to in graph[v]:

        if not bad[to]:

            bad[to] = True

            q.append(to)

answer = []

for i in range(1, n + 1):

    if dist[i] == inf:

        answer.append("*")

    elif bad[i]:

        answer.append("-")

```

```

        else:

            answer.append(str(dist[i]))

    with open("output.txt", "w", encoding="utf-8") as f:

        f.write("\n".join(answer) + "\n")

    tracemalloc.start()

    start_time = time.perf_counter()

    solve()

    finish_time = time.perf_counter()

    cur, peak = tracemalloc.get_traced_memory()

    tracemalloc.stop()

    with open("stats.txt", "w", encoding="utf-8") as f:

        f.write(f"time = {finish_time - start_time:.6f} sec\n")

        f.write(f"memory = {peak / 1024:.2f} KB\n")

```

Текстовое объяснение решения: В этой задаче нужно найти кратчайшие расстояния от одной стартовой вершины до всех остальных вершин графа. Сложность состоит в том, что веса ребер могут быть отрицательными, а также в графе могут быть отрицательные циклы.

Для решения используется алгоритм Беллмана-Форда. Сначала расстояния до всех вершин считаются бесконечными, а расстояние до стартовой вершины равно 0.

Затем выполняется $n - 1$ итерация релаксации всех ребер. Если для ребра $a \rightarrow b$ можно улучшить расстояние до вершины b , то расстояние обновляется. После этих итераций для всех вершин, которые имеют обычный кратчайший путь, будут найдены корректные расстояния.

После этого выполняется дополнительная проверка ребер. Если какое-то расстояние всё еще можно улучшить, значит эта вершина достижима из отрицательного цикла или лежит после него. Для таких вершин кратчайший путь не существует, потому что длину пути можно уменьшать бесконечно.

Но отрицательный цикл влияет не только на вершину, которая напрямую улучшается, но и на все вершины, достижимые из нее. Поэтому все такие вершины помещаются в очередь, после чего выполняется BFS по исходному графу. Все достигнутые вершины помечаются как bad.

В конце для каждой вершины выводится результат. Если вершина недостижима из стартовой, выводится *. Если вершина достижима из отрицательного цикла, выводится -. В остальных случаях выводится найденное кратчайшее расстояние.

Таблица тестирования:

время	память	input	output
0.001846 sec	11.96 KB	6 7 1 2 10 2 3 5 1 3 100 3 5 7 5 4 10 4 3 -18 6 1 -1 1	0 10 - - - *
0.001545 sec	11.53 KB	5 4 1 2 1 4 1 2 2 3 2 3 1 -5 4	0- - - 0 * *

0.001511 sec	11.58 KB	4 4 1 2 3 1 3 10 2 3 4 3 4 1 1	0 3 7 8
0.001909 sec	11.58 KB	5 3 1 2 5 2 3 -2 4 5 1 1	0 5 3 * *

Вывод по задаче

В ходе решения задачи был реализован алгоритм Беллмана-Форда для поиска кратчайших путей в графе с возможными отрицательными весами. Дополнительно была выполнена обработка вершин, на которые влияет отрицательный цикл. Решение корректно различает три случая: вершина недостижима, вершина имеет обычное кратчайшее расстояние, либо кратчайший путь для нее не существует из-за возможности бесконечного уменьшения стоимости.

Задача №11. Алхимия

Текст задачи:

Дан набор алхимических реакций. Каждая реакция превращает одно вещество в другое и записывается в формате:

вещество1 -> вещество2

Также дано исходное вещество и вещество, которое требуется получить. Нужно определить минимальное количество реакций, необходимое для получения требуемого вещества из исходного.

Если получить требуемое вещество невозможно, нужно вывести -1.

В задаче упоминается не более 100 различных веществ, а количество реакций не превосходит 1000

Код:

```
import time

import tracemalloc

from collections import deque


def get_id(name, ids):

    if name not in ids:

        ids[name] = len(ids)

    return ids[name]


def solve():

    with open("input.txt", "r", encoding="utf-8") as f:

        lines = [line.strip() for line in f.read().splitlines() if line.strip()]

    m = int(lines[0])

    ids = {}

    graph = [[] for _ in range(105)]

    for i in range(1, m + 1):

        left, right = lines[i].split(" -> ")
```

```
a = get_id(left, ids)

b = get_id(right, ids)

graph[a].append(b)

start_name = lines[m + 1]
finish_name = lines[m + 2]

start = get_id(start_name, ids)
finish = get_id(finish_name, ids)

dist = [-1] * 105

dist[start] = 0

q = deque()
q.append(start)

while q:
    v = q.popleft()

    for to in graph[v]:
        if dist[to] == -1:
            dist[to] = dist[v] + 1
            q.append(to)

with open("output.txt", "w", encoding="utf-8") as f:
```

```

        f.write(str(dist[finish]) + "\n")

    tracemalloc.start()

    start_time = time.perf_counter()

    solve()

    finish_time = time.perf_counter()

    cur, peak = tracemalloc.get_traced_memory()

    tracemalloc.stop()

    with open("stats.txt", "w", encoding="utf-8") as f:

        f.write(f"time = {finish_time - start_time:.6f} sec\n")

        f.write(f"memory = {peak / 1024:.2f} KB\n")

```

Текстовое объяснение решения: В этой задаче каждое вещество можно представить как вершину ориентированного графа, а каждую алхимическую реакцию - как направленное ребро из исходного вещества в продукт реакции.

Так как каждая реакция считается одним действием, все ребра имеют одинаковую стоимость. Поэтому минимальное количество реакций между двумя веществами можно найти с помощью поиска в ширину.

Сначала каждому названию вещества присваивается числовой номер. Это нужно, чтобы удобно хранить граф в виде списка смежности. При чтении реакции вида A -> B в граф добавляется направленное ребро из вещества A в вещество B.

После построения графа запускается BFS из исходного вещества. В массиве dist хранится минимальное количество реакций, необходимое для получения каждого вещества. Изначально расстояние до исходного вещества равно 0, а все остальные вещества имеют значение -1, что означает недостижимость.

Во время обхода, если из текущего вещества можно получить новое вещество, которое еще не посещалось, его расстояние становится на единицу больше расстояния до текущего вещества.

После завершения BFS программа выводит расстояние до требуемого вещества. Если оно осталось равно -1, значит получить это вещество невозможно.

Таблица тестирования:

время	память	input	output
0.002124 sec	11.68 KB	5 Aqua -> AquaVita AquaVita -> PhilosopherStone AquaVita -> Argentum Argentum -> Aurum AquaVita -> Aurum Aqua Aurum	2
0.001412 sec	18.88 KB	5 Aqua -> AquaVita AquaVita -> PhilosopherStone AquaVita -> Argentum Argentum -> Aurum AquaVita -> Aurum Aqua Osmium	-1
0.001598 sec	18.10 KB	3 A -> B B -> C	3

		C -> D A D	
0.001566 sec	18.14 KB	4 A -> B A -> C C -> D B -> D A D	2

Вывод по задаче

В ходе решения задачи был реализован поиск минимального количества алхимических реакций между двумя веществами. Задача была сведена к поиску кратчайшего пути в невзвешенном ориентированном графе. Для этого использовался алгоритм BFS, который гарантирует нахождение минимального количества переходов. Программа корректно обрабатывает как случай существующего преобразования, так и случай, когда получить требуемое вещество невозможно.

Общий вывод по работе

В ходе лабораторной работы были изучены и реализованы основные алгоритмы на графах: BFS, DFS, топологическая сортировка, поиск компонент сильной связности, алгоритм Дейкстры и алгоритм Беллмана-Форда.

По варианту 21 были выполнены задачи №1, №7 и №11. Для добора баллов дополнительно решены задачи №2, №3, №4, №5, №8, №9 и №10. Общая стоимость выбранных задач составляет 16.5 балла, что превышает максимум в 15 баллов.

В процессе работы были решены задачи на достижимость вершин, компоненты связности, циклы, двудольность, кратчайшие пути и отрицательные циклы. Все программы работают через файлы input.txt и output.txt, а также сохраняют время и память в stats.txt.

В результате были закреплены навыки представления графов списками смежности и применения классических алгоритмов для решения практических задач.